Churn Analysis and Customer Intelligence

In [1]:
```
# import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

1. Import database

In [2]:
```
# Data Import: db file to pandas, storing each table to a separate df

# Connect to SQLite database
conn = sqlite3.connect('customer_ch.db')

# sql query to Get all table names
sql_query = """
        SELECT name
        FROM sqlite_master
        WHERE type='table';
"""

# read sql query in pandas
tables = pd.read_sql(sql_query, conn)

# create dataframe for each table
for table_name in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table_name}", conn) # Read table into data
    globals()[f"df_{table_name}"] = df                    # Create dynamic dataf
    print(f"Created dataframe: df_{table_name}")

# Close connection
conn.close()
```

```
Created dataframe: df_db_customer
Created dataframe: df_db_subscription
Created dataframe: df_db_support
```

In [3]:
```
# Print table names and column names
conn = sqlite3.connect('customer_ch.db')

for table_name in tables['name']:
    print(f"\nTable Name: {table_name}")
    # Get column information
    columns_query = f"PRAGMA table_info({table_name});" # PRAGMA is a special co
    columns = pd.read_sql(columns_query, conn)
    print("Columns:")
    print(columns['name'].tolist())

# Close connection
conn.close()
```

```
Table Name: db_customer
Columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincod
e']

Table Name: db_subscription
Columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'p
lan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_
charges', 'cltv', 'churn_score']

Table Name: db_support
Columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

2. Data cleaning

In [4]:
```
# df_db_customer.head() # customer table
df_db_customer.head()
```

Out[4]:

	customerid	name	country	state	gender	dob	interests	pincode
0	0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
1	0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
2	0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
3	0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
4	0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

In [5]:
```
df_db_customer.tail()
```

Out[5]:

	customerid	name	country	state	gender	dob	interests	pincode
16	0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
17	0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
18	0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
19	0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
20	0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

In [6]:
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   customerid   21 non-null     str
 1   name         21 non-null     str
 2   country      18 non-null     str
 3   state        21 non-null     str
 4   gender       21 non-null     str
 5   dob          21 non-null     str
 6   interests    4 non-null      str
 7   pincode      0 non-null      object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

a. Rename Col

In [7]:
```
# a. rename col - name to customer_name

df_db_customer.rename(columns = {'name' : 'customer_name'}, inplace= True)
```

b. Drop columns

In [8]:
```
# b. drop columns - interest and pincode
# Method - 1:

# df_db_customer.drop(df_db_customer.columns[-2:], axis=1)
# df_db_customer.drop(df_db_customer.columns[6:], axis=1)

# Method 2
df_db_customer.drop(columns=['interests', 'pincode'], inplace=True)  # using col
```

In [9]:
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   customerid     21 non-null     str
 1   customer_name  21 non-null     str
 2   country        18 non-null     str
 3   state          21 non-null     str
 4   gender         21 non-null     str
 5   dob            21 non-null     str
dtypes: str(6)
memory usage: 1.1 KB
```

c. Change data type

In [10]:
```
# c. change data type - dob

df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
```

d. Data standardization

In [11]:
```
# d. data standardization - gender

# df_db_customer['gender'].unique()
df_db_customer['gender'] = df_db_customer['gender'].replace({'Men' : 'Male', 'Wo
```

In [12]:
```
df_db_customer['gender'].unique()
```

Out[12]:
```
<StringArray>
['Male', 'Female']
Length: 2, dtype: str
```

e. Fix missing values

In [13]:
```
# e. fix missing values - country

# df_db_customer['country'].isna().sum()
df_db_customer[df_db_customer['country'].isna()]
```

Out[13]:

	customerid	customer_name	country	state	gender	dob
5	0013-MHZWF	durga	NaN	Delhi	Female	1988-12-10
8	0015-UOCOJ	maya	NaN	Kathmandu	Female	1985-07-07
12	0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01

In [14]:
```
# country and state - unique value pair

# Creating state → country map from non-null rows
state_country_mapping = df_db_customer.dropna(subset=['country']).set_index('sta

# Fill the missing country using State
df_db_customer['country'] = df_db_customer['country'].fillna(df_db_customer['sta
```

In [15]:
```
df_db_customer[df_db_customer['country'].isna()] # no null value in country col
```

Out[15]:

	customerid	customer_name	country	state	gender	dob

In [16]:
```
# --------- Next Table -----------
```

In [17]:
```
df_db_subscription.head() # subcription table
```

Out[17]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	

In [18]:
```
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   customerid              21 non-null     str
 1   subscription_start_date 21 non-null     str
 2   subscription_type       21 non-null     str
 3   renewal_date            21 non-null     str
 4   plan_type               21 non-null     str
 5   contract_type           21 non-null     str
 6   cancellation_date       6 non-null      str
 7   cancellation_reason     6 non-null      str
 8   monthly_charges         21 non-null     float64
 9   cltv                    21 non-null     int64
 10  churn_score             21 non-null     int64
dtypes: float64(1), int64(2), str(8)
memory usage: 1.9 KB
```

In [19]:
```
# change data type to date - subscription_start_date , renewal_date, cancellatio
date_col = ['subscription_start_date', 'renewal_date', 'cancellation_date']

df_db_subscription[date_col] = df_db_subscription[date_col].apply(pd.to_datetime
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   customerid               21 non-null     str
 1   subscription_start_date  21 non-null     datetime64[us]
 2   subscription_type        21 non-null     str
 3   renewal_date             21 non-null     datetime64[us]
 4   plan_type                21 non-null     str
 5   contract_type            21 non-null     str
 6   cancellation_date        6 non-null      datetime64[us]
 7   cancellation_reason      6 non-null      str
 8   monthly_charges          21 non-null     float64
 9   cltv                     21 non-null     int64
 10  churn_score              21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB
```

In [20]:
```
# -------- NEXT TABLE -----------------
```

In [21]:
```
df_db_support.head() # support table
```

Out[21]:

	customerid	complaint_date	escalations	csat_score	col_1	comment
0	0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
1	0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
2	0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
3	0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
4	0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN

In [22]:
```
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      str
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
 4   col_1           0 non-null      object
 5   comment         4 non-null      str
dtypes: int64(1), object(1), str(4)
memory usage: 564.0+ bytes
```

In [23]:
```
# drop columns
df_db_support.drop(columns=['col_1', 'comment'], inplace=True)
```

In [24]:
```
# change data type to date - complaint_date

df_db_support['complaint_date'] = pd.to_datetime(df_db_support['complaint_date']
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      datetime64[us]
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 420.0 bytes
```

In [ ]:
```

```

3. Feature Engineering & Data Analysis

Create a new for churn flag -> if a person has churn or not

In [25]:
```
# create a new col using existing col - churn flag

# Customer is churned if cancellation_date is not null
df_db_subscription['churn_flag'] = np.where(df_db_subscription['cancellation_dat
df_db_subscription.head()
```

Out[25]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	

In [ ]:

Merge Dataframes - JOINs

In [26]:
```
# Note: while merging df's always check the shape before and after

df = (df_db_subscription
            .merge(df_db_customer, on = 'customerid', how= 'left')
            .merge(df_db_support, on = 'customerid', how= 'left') )
```

In [27]:
```
df_db_subscription.shape
```

Out[27]: (21, 12)

In [28]:
```
df.shape # Our actual data has 21 rows but here 23 rows are coming
```

Out[28]: (23, 20)

In [29]:
```
df.head() # we see here two rows are nearly same
```

Out[29]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
3	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
4	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	

In [30]:
```
print('df_db_subscription unique value:', df_db_subscription['customerid'].nuniq
print('df_db_customer unique value:', df_db_customer['customerid'].nunique())
```

```
print('df_db_support unique value:', df_db_support['customerid'].nunique())
print('df_db_support all value:', df_db_support['customerid'].size)
```

```
df_db_subscription unique value: 21
df_db_customer unique value: 21
df_db_support unique value: 7
df_db_support all value: 9
```

In [31]:
```
df_db_support #Same cutomer but different reports
```

Out[31]:

	customerid	complaint_date	escalations	csat_score
0	0003-MKNFE	2024-08-28	N	60
1	0003-MKNFE	2024-08-28	Y	10
2	0013-EXCHZ	2024-01-20	Y	20
3	0013-MHZWF	2025-03-18	N	90
4	0013-SMEOE	2024-11-01	N	30
5	0017-IUDMW	2024-04-10	Y	25
6	0019-EFAEP	2024-09-27	Y	30
7	0022-TCJCI	2024-09-13	Y	10
8	0022-TCJCI	2024-09-14	N	90

In [32]:
```
# Creating a new Column for count
df_db_support['complaint_count'] = df_db_support.groupby('customerid')['customer
```

In [33]:
```
# droping duplicates in terms of cutomer id
df_db_support = df_db_support.sort_values('complaint_date').drop_duplicates('cus
```

In [35]:
```
df_db_support['customerid'].size
```

Out[35]: 7

In [36]:
```
# merge df (As we have fixed the support table duplicates)
df = (df_db_subscription
            .merge(df_db_customer, on = 'customerid', how= 'left')
            .merge(df_db_support, on = 'customerid', how= 'left') )
```

In [37]:
```
df.shape
```

Out[37]: (21, 21)

Export dataframe

In [38]:
```
# export dataframe to csv file
df.to_csv('exported_churn_data.csv', index=False)
```

In [ ]:
```

```

Data Analysis

In [39]:
```
# 1. Churn Rate
```

In [40]:
```
df.columns
```

Out[40]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='str')
```

In [41]:
```
# Percentage of churn people
churn_rate = df['churn_flag'].mean()*100
print("Churn Rate = ", round(churn_rate,2), "%")
```

```
Churn Rate =  28.57 %
```

In [42]:
```
# 2. Retenion Rate (Opposite of vhurn rate)
retention_rate = 100 - churn_rate
print("Retention Rate = ", round(retention_rate,2), "%")
```

```
Retention Rate =  71.43 %
```

In [43]:
```
df.head() # we want to see which people are going to churn
```

Out[43]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	

5 rows × 21 columns

In [44]:
```
# 3. Churn by Plan type
churn_by_plan = (df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).
print(churn_by_plan)
```

```
  plan_type  churn_rate_pct
0     Basic           60.00
1   Premium           14.29
2  Standard           22.22
```

In [45]:
```
# 4 a. Churn by state + sum(revenue) & count of users
churn_by_state = (df.groupby('state').agg(total_revenue=('monthly_charges', 'sum

churn_by_state
```

Out[45]:

	state	total_revenue	total_users	churned_users
0	Delhi	52.96	4	1
1	Karnataka	20.98	2	2
2	Kathmandu	20.98	2	0
3	Maharashtra	50.97	3	0
4	Meghalaya	42.97	3	2
5	Nagaland	22.99	1	0
6	Rajasthan	36.98	2	0
7	Telangana	30.98	2	1
8	Uttar Pradesh	115.98	2	0

In [47]:
```
# 4 b. Churn by subscription type + sum(revenue) & count of users
churn_by_subscription = df.groupby('subscription_type').agg(total_revenue=('mont
churn_by_subscription
```

Out[47]:

	subscription_type	total_revenue	total_users	churned_users
0	Organic	145.91	9	0
1	Paid	174.94	6	1
2	Refferal	74.94	6	5

In [48]:
```
# 5. ARPU - Avg Revenue per user
arpu = df['monthly_charges'].mean()
print('ARPU = ', round(arpu,2))
```

```
ARPU =  18.85
```

In [50]:
```
from dateutil.relativedelta import relativedelta; df['age'] = pd.to_datetime(df[
```

In [51]:
```
df[['dob', 'age']].head()
```

Out[51]:

	dob	age
0	1982-04-12	44
1	1995-11-23	30
2	1978-02-15	48
3	2001-08-30	24
4	1990-05-05	36

In [52]:
```
# 6. Avg Customer Tenure
# count of days users has used our service : cancellation date else current date
today = pd.Timestamp.today()

df['tenure_days'] = np.where(
        df['cancellation_date'].notna(),
```

```
    (df['cancellation_date'] - df['subscription_start_date']).dt.days,

    (today - df['subscription_start_date']).dt.days
)

avg_tenure = df['tenure_days'].mean()
print("Avg Tenure (Days) = ", round(avg_tenure),0)
```

```
Avg Tenure (Days) =  1507 0
```

In [53]:
```
# 7. Revenue at risk - revenue lost from churned users
revenue_at_risk = df.loc[df['churn_flag']==1, 'monthly_charges'].sum()
print("Revenue at Risk (Rs 'K') =", revenue_at_risk)
```

```
Revenue at Risk (Rs 'K') = 73.94
```

In [54]:
```
# 8. Esclation Rate
escalation_rate = (df['escalations']=='Y').mean()*100
print("Esclation Rate = ", round(escalation_rate, 2), "%")
```

```
Esclation Rate =  19.05 %
```

In [55]:
```
# 9. Avg Complaint Per User
avg_complaints = df['complaint_count'].sum() / df['customerid'].nunique()
print("Avg Compliants Per User = ", round(avg_complaints, 2))
```

```
Avg Compliants Per User =  0.43
```

In [56]:
```
# 10. Correlation Esclation vs Churn
df['escalations'] = np.where(df['escalations'] == 'Y', 1, 0) # encoding string t
corr_df = df[['escalations', 'churn_flag']].dropna()

correlation = corr_df['escalations'].corr(df['churn_flag'])
print("Correlation between esclation vs churn is = ", round(correlation,2))
```

```
Correlation between esclation vs churn is =  0.77
```

In [57]:
```
# 11. Create a column using existing col - Churn risk
conditions = [
        (df['churn_score'] < 50),
        (df['churn_score'] >= 50) & (df['churn_score'] < 70),
        (df['churn_score'] >= 70)
]

choices = ['low', 'med', 'high']

df['churn_risk'] = np.select(conditions, choices, default='unkown')
```

In [58]:
```
df[['churn_risk', 'churn_score']].head()
```

Out[58]:

	churn_risk	churn_score
0	low	12
1	high	91
2	low	34
3	low	8
4	high	88

In [ ]:

4. Visualization using Matplotlib

In [59]:
```
df.head()
```

Out[59]:

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	cont
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	

5 rows × 24 columns

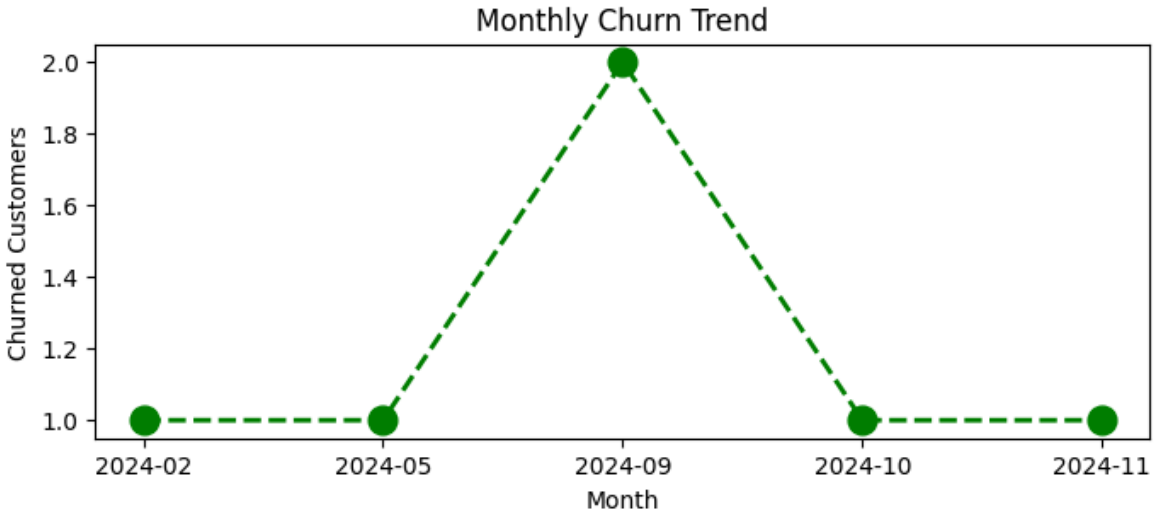
In [60]:
```
# best practice to create a copy then work on it
df_visual = df.copy()
```

In [61]:
```
df_visual.columns
```

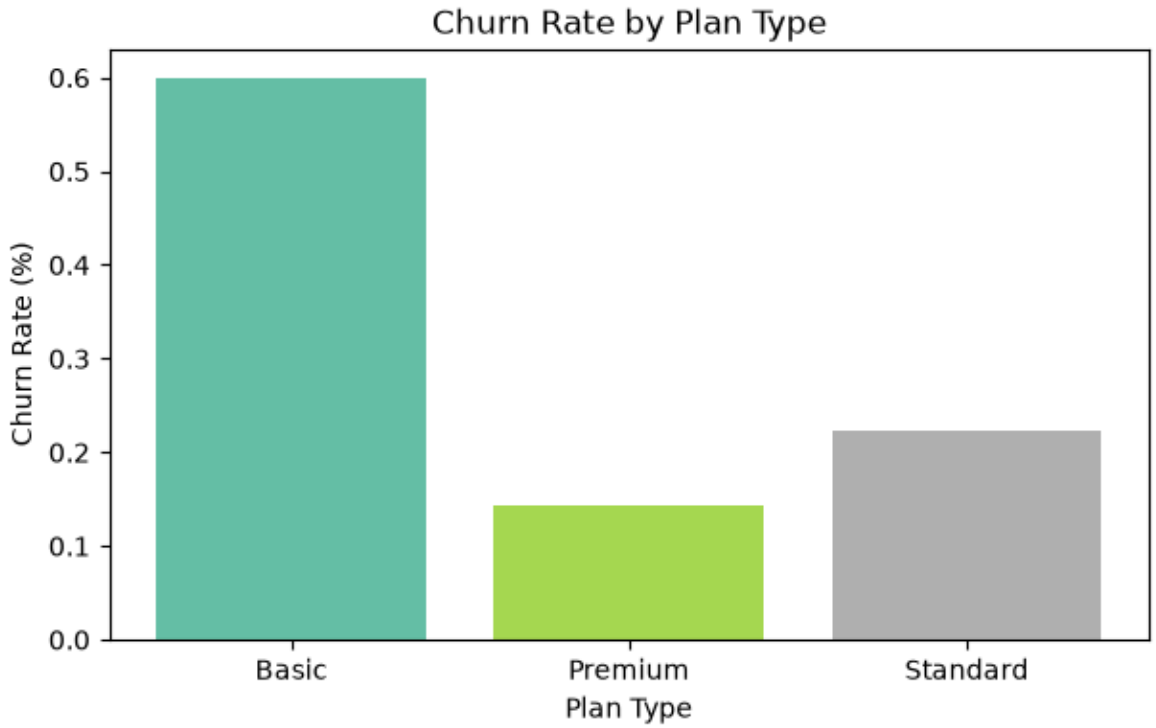
Out[61]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'age',
       'tenure_days', 'churn_risk'],
      dtype='str')
```

In [ ]:
```
# 4.1 Monthly Churn Trend (Time Series KPI)

df_visual['cancellation_month'] = df_visual['cancellation_date'].dt.to_period('M

churn_trend = df_visual[df_visual['churn_flag'] == 1].groupby('cancellation_mont
```

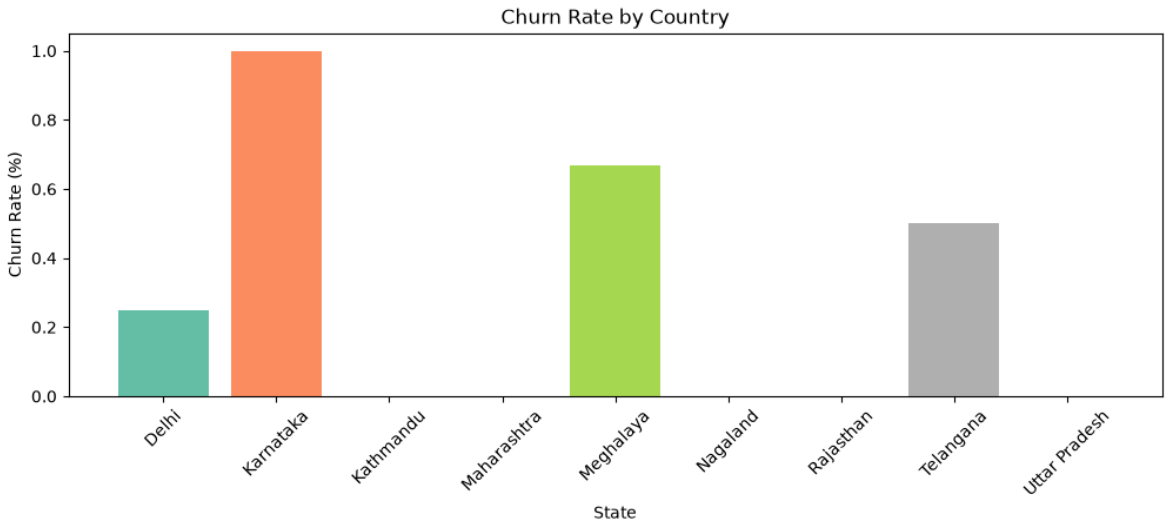
```
plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str), churn_trend.values,  color='green', mark

plt.title('Monthly Churn Trend')
plt.xlabel('Month')
plt.ylabel('Churned Customers')
plt.show()
```



In [62]:
```
# 4.2 Churn Rate by Plan type
churn_plan = df_visual.groupby('plan_type')['churn_flag'].mean()

colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan))) # it will make color ac

plt.figure(figsize=(7,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Plan Type')
plt.xlabel('Plan Type')
plt.ylabel('Churn Rate (%)')
plt.show()
```



In [63]:
```
# 4.3 Churn by States
churn_plan = df_visual.groupby('state')['churn_flag'].mean()

# colors = ['yellow', 'purple', 'blue']
colors = plt.cm.Set2(np.linspace(0, 1, len(churn_plan)))

plt.figure(figsize=(12,4))
plt.bar(churn_plan.index, churn_plan.values, color = colors)

plt.title('Churn Rate by Country')
plt.xlabel('State')
plt.ylabel('Churn Rate (%)')
plt.xticks(rotation=45)
plt.show()
```



In [ ]:
```
```

Visulaization using Seaborn

In [64]:
```
# Heatmap (Correlation Matrix)
```

In [65]:
```
# encoding - convert str to numeric so that we can find corr between features
df_visual.columns
```

Out[65]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'age',
       'tenure_days', 'churn_risk'],
      dtype='str')
```

In [66]:
```
df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_ris
```

Out[66]:

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	Standard	Annual	12	0	low	0
1	Premium	Annual	91	1	high	1
2	Basic	Monthly	34	0	low	0
3	Premium	Annual	8	0	low	0
4	Standard	Monthly	88	1	high	1

In [67]:
```
# remove warnings
import warnings
warnings.filterwarnings("ignore")
```

In [68]:
```
# incorrect method of encoding - as numbers are not assigned based on priority
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag

categorial_cols = ['plan_type', 'contract_type', 'churn_risk']

for col in categorial_cols:
    df_encoded[col] = df_encoded[col].astype('category').cat.codes
```

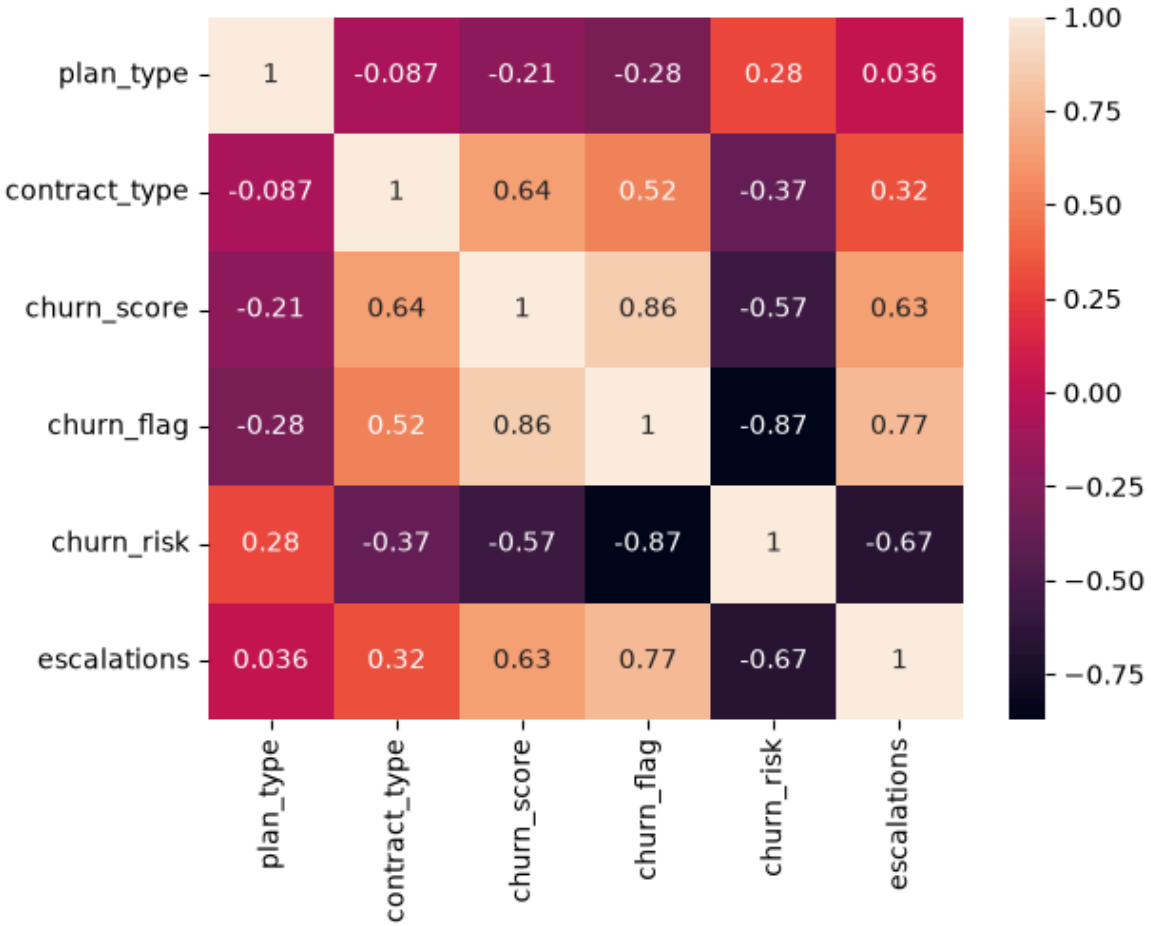
In [69]:
```
df_encoded.head()
```

Out[69]:

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	2	0	12	0	1	0
1	1	0	91	1	0	1
2	0	1	34	0	1	0
3	1	0	8	0	1	0
4	2	1	88	1	0	1

In [70]:
```
# Heatmap (correlation matrix)

sns.heatmap(df_encoded.corr(), annot=True)
```

Out[70]: <Axes: >



In [72]:
```
# here we see that the realtion between the escalation abd the churn risk is goo

# for churn_risk -> low - 1, medium -> 2 , high -> 0 as per alphabetical order t
```

In [73]:
```
print('plan_type :', df_visual['plan_type'].unique())
print('contract_type :', df_visual['contract_type'].unique())
print('churn_risk :', df_visual['churn_risk'].unique())
```

```
plan_type : <StringArray>
['Standard', 'Premium', 'Basic']
Length: 3, dtype: str
contract_type : <StringArray>
['Annual', 'Monthly']
Length: 2, dtype: str
churn_risk : <StringArray>
['low', 'high', 'med']
Length: 3, dtype: str
```

In [74]:
```
# Correct method of encoding - based on priority
df_encoded = df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag

order_mappings = {
    'plan_type' : ['Basic', 'Standard', 'Premium'],
    'contract_type' : ['Monthly', 'Annual'],
    'churn_risk': ['low', 'med', 'high']
    }

for col, order in order_mappings.items():
    df_encoded[col] = pd.Categorical(df_encoded[col].astype('category'), categor
```

In [77]:
```
df_visual[['plan_type', 'contract_type', 'churn_score', 'churn_flag', 'churn_ris
```

Out[77]:

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	Standard	Annual	12	0	low	0
1	Premium	Annual	91	1	high	1
2	Basic	Monthly	34	0	low	0
3	Premium	Annual	8	0	low	0
4	Standard	Monthly	88	1	high	1

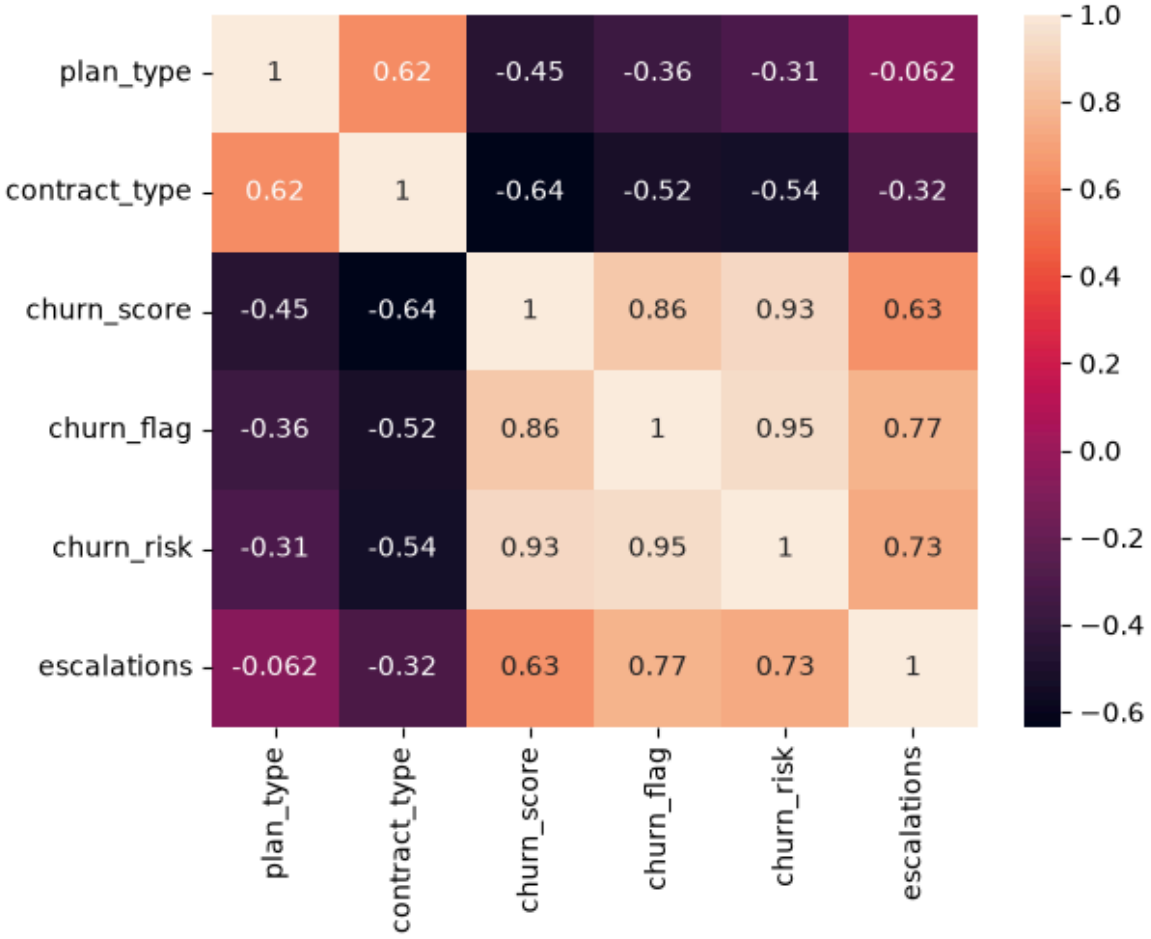
In [78]:
```
df_encoded.head()
```

Out[78]:

	plan_type	contract_type	churn_score	churn_flag	churn_risk	escalations
0	1	1	12	0	0	0
1	2	1	91	1	2	1
2	0	0	34	0	0	0
3	2	1	8	0	0	0
4	1	0	88	1	2	1

In [79]:
```
# Heatmap (correlation matrix)
sns.heatmap(df_encoded.corr(), annot=True)
```

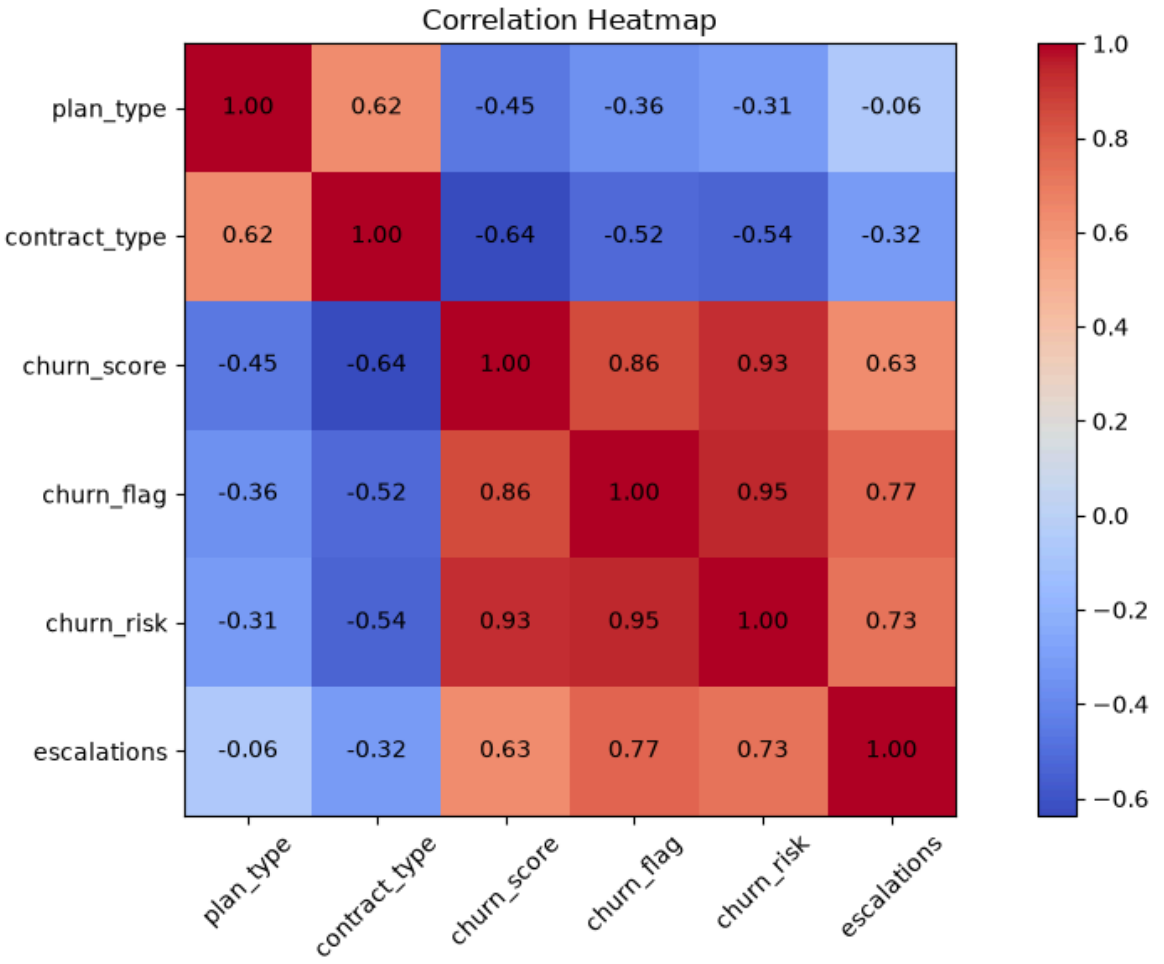
Out[79]:
```
<Axes: >
```



In [80]:

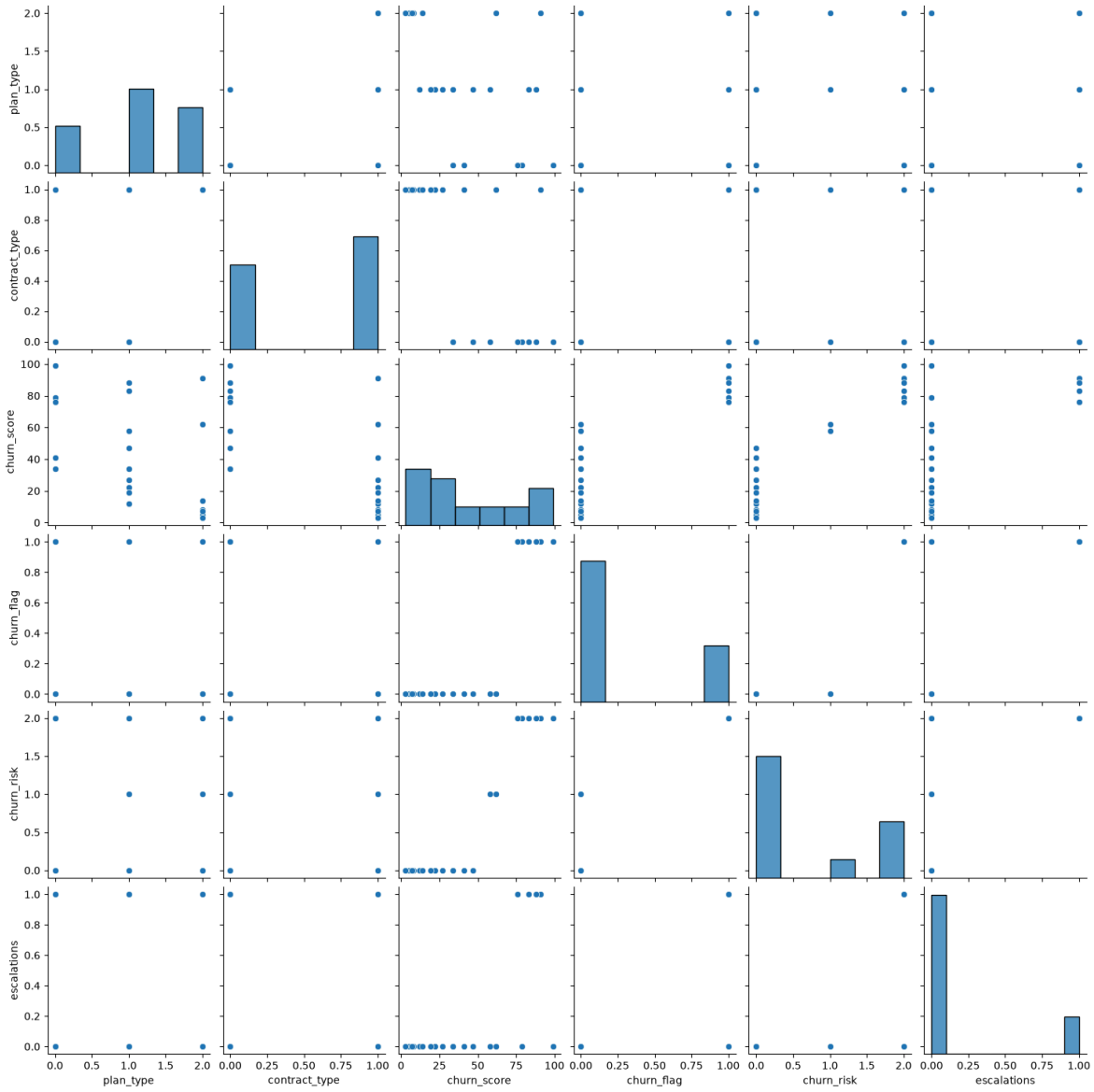
```
# Heatmap Using Matplotlib - difficult to plot it

corr_matrix = df_encoded.corr() # Correlation matrix

fig, ax = plt.subplots(figsize=(10, 6)) # Create figure

cax = ax.imshow(corr_matrix, cmap='coolwarm') # Heatmap

fig.colorbar(cax) # Add colorbar

# Axis labels
ax.set_xticks(np.arange(len(corr_matrix.columns)))
ax.set_yticks(np.arange(len(corr_matrix.columns)))

ax.set_xticklabels(corr_matrix.columns, rotation=45)
ax.set_yticklabels(corr_matrix.columns)

# Annotate values inside cells
for i in range(len(corr_matrix.columns)):
    for j in range(len(corr_matrix.columns)):
        ax.text(
            j,
            i,
            f"{corr_matrix.iloc[i, j]:.2f}",
            ha='center',
            va='center'
        )

plt.title('Correlation Heatmap') # Title
```

```
plt.tight_layout()
plt.show()
```



In [81]:
```
# pairplot - Plot pairwise relationships in a dataset
sns.pairplot(df_encoded)
```

Out[81]: <seaborn.axisgrid.PairGrid at 0x18b75f2dfd0>

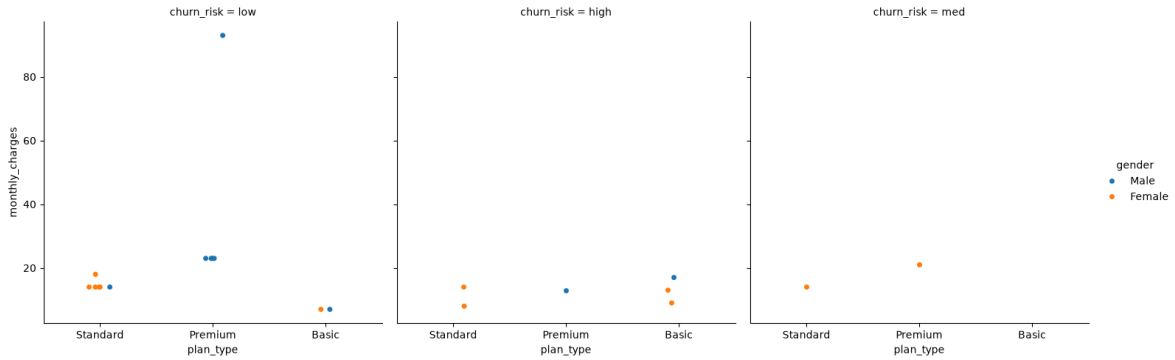


In [82]:
```
df_visual.columns
```

Out[82]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count', 'age',
       'tenure_days', 'churn_risk'],
      dtype='str')
```

In [83]:
```
# catplt/Facegrid plot - multi-dim comparison

sns.catplot(data=df_visual,
    x='plan_type',
    y='monthly_charges',
    hue='gender',
    col='churn_risk')
```

Out[83]:
```
<seaborn.axisgrid.FacetGrid at 0x18b776fd6a0>
```



In [ ]:

Pivot table

In [84]:
```
# Pivot Table

pd.pivot_table(
    df_visual,
    index='plan_type',
    values='churn_flag',
    aggfunc = 'mean'
)
```

Out[84]:

plan_type	churn_flag
Basic	0.600000
Premium	0.142857
Standard	0.222222

In [85]:
```
# pivot table using multiple cols and agg type

pd.pivot_table(
    df_visual,
    index='plan_type',
    values=['monthly_charges', 'customerid', 'churn_flag'],
    aggfunc = {
        'monthly_charges' : 'sum',
        'customerid' : 'nunique',
        'churn_flag' : 'mean'
    }
)
```

Out[85]:

plan_type	churn_flag	customerid	monthly_charges
Basic	0.600000	5	52.95
Premium	0.142857	7	218.93
Standard	0.222222	9	123.91

In [ ]:

Working with SQL in Python

In [86]:
```
# create db in sql
conn = sqlite3.connect('test_database.sqlite')

#table details
conn.execute("CREATE TABLE users (first_name TEXT, country TEXT, budget INTEGER)")

# commit and save
conn.commit()

print("Created Table successfully!")
```

```
Created Table successfully!
```

In [87]:
```
# insert data

cursor = conn.cursor()

# write sql query to insert records in sql table
cursor.execute(
    """
        INSERT INTO users VALUES
            ('Madhav', 'India', 5000),
            ('Rishabh', 'Germany', 2500),
            ('Vishakha', 'India', 3500)
    """
)

# commit and save
conn.commit()

print("Data Inserted successfully!")
```

```
Data Inserted successfully!
```

In [88]:
```
# check inserted data in table
conn = sqlite3.connect('test_database.sqlite')
query = """SELECT * FROM users"""

df_results = pd.read_sql(query, conn)

df_results.head()
```

Out[88]:

	first_name	country	budget
0	Madhav	India	5000
1	Rishabh	Germany	2500
2	Vishakha	India	3500

In [89]:
```
# aggregation
query = """
        SELECT country, SUM(budget) as total_budget
        FROM users
```

```
        GROUP BY country
"""
df_agg = pd.read_sql(query, conn)
df_agg
```

Out[89]:

	country	total_budget
0	Germany	2500
1	India	8500

In [ ]:

```
#Close the connections
conn.close()
```